

Reviewer Pack — Minimal Ehrhart Quotient and Circuit Monotone Width Inequality...

Entry 24f15defb69f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 02:20:57 UTC

Minimal Ehrhart Quotient and Circuit Monotone Width Inequality

Entry ID: 24f15defb69f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 02:20:57 UTC

1. Conjecture

Field A (mathematical branch): Polyhedral Geometry (Ehrhart Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every CNF φ with n variables, the minimal Ehrhart quotient $\mu(\varphi)$ of its incidence polytope is linearly correlated with its circuit monotone width $w_m(\varphi)$, such that $\mu(\varphi) = O(w_m(\varphi))$ and $\mu(\varphi)^2 \leq w_m(\varphi)$.

Rationale (proposer's reasoning):

Ehrhart theory studies the divisibility properties of lattice points in polytopes, which can be related to the number of ways a boolean function can be represented. The monotone width of a circuit is a measure of its complexity. This conjecture explores whether Ehrhart's theorem, traditionally used in combinatorial and arithmetic geometry, can shed light on the complexity of circuits.

Taxonomy category: PolyhedralGeometry × BooleanCircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9827df8a77126774

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the seeds have a Pearson correlation coefficient between $\mu(\varphi)$ and $w_m(\varphi) \geq 0.5$ and satisfy $\mu(\varphi)^2 \leq w_m(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Ehrhart quotient" AND "circuit monotone width" - "incidence polytope" AND minimal Ehrhart quotient" AND circuit complexity" - "CNF" AND Ehrhart theory" AND circuit monotone width inequality"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    else:
        g, x, y = extended_gcd(b % a, a)
        return g, y - (b // a) * x, x

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % m

def matrix_multiplication(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])
```

```

if cols_A != rows_B:
    raise ValueError("Matrix dimensions do not match")

result = [[sum(A[i][k] * B[k][j] for k in range(cols_A)) for j in range(cols_B)] for i in range(rows_A)]
return result

def gaussian_elimination(matrix):
    n = len(matrix)
    m = len(matrix[0])
    augmented_matrix = [row + [1 if i == j else 0 for j in range(m, 2*m)] for i, row in enumerate(matrix)]

    for i in range(n):
        max_row = i
        for k in range(i+1, n):
            if abs(augmented_matrix[k][i]) > abs(augmented_matrix[max_row][i]):
                max_row = k

        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        factor = augmented_matrix[i][i]
        for j in range(i, 2*m):
            augmented_matrix[i][j] /= factor

        for k in range(n):
            if k != i:
                factor = augmented_matrix[k][i]
                for j in range(i, 2*m):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]

    return [row[m:] for row in augmented_matrix]

def min_ehrhart_quotient(cnf):
    n = len(cnf)
    variables = set(var for clause in cnf for var in clause if isinstance(var, int))
    variables = sorted(variables)

    polytope_points = []
    for assignment in product([-1, 1], repeat=n):
        if all(any(sign * var == assignment[var-1] for sign, var in clause) for clause in cnf):
            point = [0] * (2*n)
            for i, var in enumerate(variables):
                point[i] = assignment[i]
                point[n+i] = -assignment[i]
            polytope_points.append(point)

    if not polytope_points:
        return 0

    matrix = []
    for point in polytope_points:
        row = [point[i] * point[j] for i in range(2*n) for j in range(2*n)]
        matrix.append(row)

    reduced_matrix = gaussian_elimination(matrix)
    rank = sum(1 for row in reduced_matrix if any(x != 0 for x in row))

```

```

    return len(polytope_points) // rank

def circuit_monotone_width(cnf):
    n = len(cnf)
    variables = set(var for clause in cnf for var in clause if isinstance(var, int))
    variables = sorted(variables)

    monotone_width = 0
    for assignment in product([-1, 1], repeat=n):
        if all(any(sign * var == assignment[var-1] for sign, var in clause) for clause in cnf):
            monotone_width += 1

    return monotone_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_random_cnf(n)

    mu = min_ehrhart_quotient(cnf)
    wm = circuit_monotone_width(cnf)

    if mu == 0 or wm == 0:
        return {
            "metric_name": "min_ehrhart_quotient",
            "metric_value": mu,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    correlation = (mu * wm) / (math.sqrt(mu**2 + wm**2))
    holds_bound = mu**2 <= wm

    return {
        "metric_name": "min_ehrhart_quotient",
        "metric_value": mu,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": correlation >= 0.5 and holds_bound,
        "counterexample": ""
    }

def generate_random_cnf(n):
    num_clauses = random.randint(2*n, 3*n)
    variables = list(range(1, n+1))
    cnf = []

    for _ in range(num_clauses):
        clause = []
        while not clause:
            literals = random.sample(variables + [-var for var in variables], n)
            clause = [(l if l > 0 else -l, abs(l)) for l in literals]
        cnf.append(clause)

```

```

    return cnf

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j + 5**k for i, j, k in product(range(5),
                                                range(5), range(5))]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=' ' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5bc32cfb.py", line 165, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5bc32cfb.py", line 120, in run_trial
    mu = min_ehrhart_quotient(cnf)
         ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5bc32cfb.py", line 82, in min_ehrhart_q
    for assignment in product([-1, 1], repeat=n):
                       ^^^^^^^
NameError: name 'product' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Review and fix the error in the test code that caused it to crash, then rerun the test with a larger sample size to ensure the results are reliable.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12269
2	propose	ollama_remote	glm4:latest	0	0	14465
3	preregistration	ollama_remote	glm4:latest	0	0	9434
4	novelty	ollama_remote	glm4:latest	0	0	8354
5	novelty	ollama_remote	glm4:latest	0	0	9045
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21911
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15398
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36737
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20519
10	judge	ollama_remote	glm4:latest	0	0	66825

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 214958 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/24f15defb69f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/24f15defb69f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/24f15defb69f.tar.gz* (if generated)